

DataLab Lab1 实验报告

姓名：陈睿尹 学号：25800190016

10/05

1 实验结果

1.1 运行 `./check_ops.py bits.c`

在终端中运行：

```
./check_ops.py bits.c
```

最终 19 个函数均通过操作数检查。

```
root@ics:~/MyLab1# ./check_ops.py
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 6/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 14/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 29/32 operators
isBetweenEitherOrder: 35/48 operators
mul5Sat: 22/30 operators
classifyAdd3: 36/52 operators
floatScaleThreeHalves: 45/60 operators
floatRoundEven: 35/65 operators
float_i2f: 37/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

Figure 1: `./check_ops.py bits.c` 的运行结果

1.2 运行 `./btest`

在终端中运行：

```
./btest
```

最终所有测试均通过，结果为：

Total points: 110/110

```
root@ics:~/MyLab1# make clean
rm -f *.o btest fshow ishow *~
root@ics:~/MyLab1# make all
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
root@ics:~/MyLab1# ./btest
Score Rating Errors Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

Figure 2: ./btest 的运行结果

2 各函数实现思路

2.1 P1 signMask

题目要求：返回一个只有最高位为 1、其余位均为 0 的 32 位整数。

最高位是第 31 位，因此直接将 1 左移 31 位：

```
int signMask(void) {
    return 1 << 31;
}
```

2.2 P2 bitXor

题目要求：只能使用规定的运算符实现异或运算。

分别构造“只有 $x = 1, y = 0$ ”和“ $x = 0, y = 1$ ”的情况，再将两种情况合并：

```
int bitXor(int x, int y) {
    return ~(~(x & ~y) & ~(~x & y));
}
```

2.3 P3 negativePart

题目要求：当 x 为负数时返回 $-x$ ，否则返回 0。

利用最高位得到符号信息；如果为负数，则通过补码得到相反数：

```
int negativePart(int x){
    return ~((((1<<31)&x)>>31)&x)+1;
}
```

2.4 P4 copyByteWithin

题目要求：将 x 中第 src 个字节复制到第 dst 个字节位置。

先用 0xFF 提取源字节，再移动到目标位置，同时清除目标位置原来的内容：

```
return ((0xFF&(x>>(src<<3))) << (dst<<3))
    | ~(0xFF<<(dst<<3)) & x;
```

2.5 P5 logicalShift

题目要求：实现逻辑右移，即右移后左侧补 0。

由于有符号数右移可能进行算术右移，因此先进行右移，再用掩码去掉左侧补上的符号位：

```
int logicalShift(int x, int n) {
    return ~(1<<31>>n<<1) & (x>>n);
}
```

2.6 P6 swapNibblePairs

题目要求：将每个字节中的高 4 位和低 4 位交换。

构造每个字节低 4 位为 1 的掩码，分别取出两个 nibble 后移动并重新组合：

```
a=0xF;
b=a+(a<<8);
c=b+(b<<16);
d=~c;

return ((x&c)<<4) |
    (((x&d)>>4)&(~(1<<31>>3)));
```

2.7 P7 secondLowestZeroBit

题目要求：找出从最低位开始第二个值为 0 的 bit，并返回对应的掩码。

利用 $x + 1$ 与 x 的组合找到最低位 0，再重复一次得到第二个 0：

```
int a=(x+1)|x;
return ((a+1)|a)^a;
```

2.8 P8 oddParity

题目要求：判断一个整数中 1 的个数是奇数还是偶数。

通过连续异或逐渐压缩 32 位信息，最后只检查最低位：

```
x=x^(x>>16);
x=x^(x>>8);
x=x^(x>>4);
x=x^(x>>2);
x=x^(x>>1);

return !(x&1);
```

2.9 P9 rotateRightBits

题目要求：将一个 32 位整数循环右移 n 位。

普通右移得到右侧部分，被移出的低位需要重新移动到最高位，因此分别计算两部分后进行或运算：

```
int a=(~(1<<31>>n<<1)) & (x>>n);
int b=x<<1<<(31+~n+1);

return a|b;
```

其中第一部分还需要处理算术右移产生的符号扩展。

2.10 P10 roundEvenPow2

题目要求：将整数舍入到最接近的 2^n 的倍数，并在正好处于中间时取偶数。

先得到商和余数：

```
q = x >> n;
r = (~(1 << 31 >> (32 + ~n))) & x;
```

然后根据余数是否超过一半，以及恰好一半时商的奇偶性判断是否进位：

```

sm = 1 << n >> 1;
ex = (r & sm) << 1;

return (q << n) + ex
       + (~(((!(r ^ sm)) & !(q & 1)) << n) + 1);

```

关键是处理题目要求的 round-to-nearest-even。

2.11 P11 midpointTowardFirst

题目要求：求 x 和 y 的中点，并在不能精确表示时向第一个数方向取整。

直接计算 $x + y$ 可能溢出，因此先分别右移，再根据符号和最低位确定是否需要补 1：

```

p1=x>>1;
p2=y>>1;

int chk=sx^sy;

pd=((!chk)&(!((1<<31)&(x+~y+1))))
   +(chk&(!((1<<31)&(p1+~p2+1))));

ex=(x&y&1)+(((1&x)^(1&y))&pd);

```

这样可以避免直接计算 $x + y$ 带来的溢出。

2.12 P12 isBetweenEitherOrder

题目要求：判断 x 是否位于 a 和 b 之间，且不要求 $a < b$ 。

分别计算 $x - a$ 和 $x - b$ ，再结合三个数的符号判断位置：

```

int da=x+~a+1, db=x+~b+1;
int ca=sx^sa, cb=sx^sb;

int sda=((ca&sx)|((!ca)&((da>>31)&1)));
int sdb=((cb&sx)|((!cb)&((db>>31)&1)));

```

最后还需要处理 x 等于任意一个端点的情况：

```

return !((sda^sdb)|(!(x^a))|(!(x^b)));

```

2.13 P13 mul5Sat

题目要求：计算 $5x$ ，如果发生溢出则返回对应的最大值或最小值。

先通过左移得到 $4x$ ，再加上 x ：

```
int y=x<<2;
int z=x+y;
```

然后利用符号位判断移位和加法过程中是否发生溢出：

```
int shov=!!(x^(y>>2));
int ov=(~(x^y))&(x^z);
```

最后根据溢出情况选择正常结果或饱和值：

```
int m=(~shov+1)|(ov>>31);
int sat=(x>>31)^~(1<<31);

return (z&~m)|(sat&m);
```

2.14 P14 classifyAdd3

题目要求：判断数学意义上的 $x + y + z$ 是正数、0 还是负数，同时处理整数溢出。

将计算拆成两次加法，并分别判断正溢出和负溢出：

```
int s=x+y;
int ss=s>>31;

int po=(~(sx|sy))&ss;
int ne=sx&sy&~ss;
```

然后对第二次加法进行同样处理，最后将两次结果组合：

```
int H=h+k, sh=H>>31;
return sh|((!sh)&!!H);
```

核心思想是不能简单依赖最终的 32 位结果，而需要分析加法过程中是否发生溢出。

2.15 P15 floatScaleThreeHalves

题目要求：将 IEEE 754 单精度浮点数乘以 $3/2$ ，并返回对应的 bit pattern。

首先拆分符号位、指数和 fraction：

```
unsigned s=uf&0x80000000;
unsigned e=(uf>>23)&0xFF;
unsigned f=uf&0x7FFFFFFF;
```

NaN 和无穷大直接返回：

```
if(e==0xFF)
    return uf;
```

对于非规格化数，直接处理 fraction：

```
unsigned t=f*3;
unsigned r=t>>1;

if((t&1)&&(r&1))
    r++;
```

对于规格化数，需要把隐藏的最高位补回来：

```
unsigned t=(f|0x800000)*3;
```

随后根据有效数字的范围调整移位，并处理舍入和指数溢出。

这一题的主要难点是同时处理规格化数、非规格化数、舍入以及无穷大的情况。

2.16 P16 floatRoundEven

题目要求：将单精度浮点数舍入到最接近的整数，并使用 round-to-even。

首先处理特殊值以及已经是整数的情况：

```
if(e==0xFF)
    return uf;

if(e>=150)
    return uf;

if(e<126)
    return s;
```

对于需要舍入的数，恢复隐藏位并取得整数部分：

```
sig=f|0x800000;
sh=150-e;
q=sig>>sh;
```

再根据被舍弃部分决定是否进位：

```
mask=(1<<sh)-1;
r=sig&mask;

if(r>(1<<(sh-1)) ||
    (r==(1<<(sh-1))&&(q&1)))
    q++;
```

这里的关键是正确处理恰好位于两个整数中间的情况。

2.17 P17 float_i2f

题目要求：将 32 位整数转换成 IEEE 754 单精度浮点数的 bit pattern。

先处理 0 和负数：

```
if(x==0)
    return 0;

if(x<0){
    s=0x80000000;
    a=~x+1;
}
else
    a=x;
```

然后通过不断右移找到最高位：

```
while(a>>1){
    a>>=1;
    e++;
}
```

如果有效位超过 23 位，就需要舍弃低位并进行 round-to-even：

```
r=f&((1<<sh)-1);
f=(f>>sh)&0x7FFFFFFF;

if(r>(1<<(sh-1)) ||
    (r==(1<<(sh-1))&&(f&1))){
    f++;
    if(f==0x800000){
        f=0;
        e++;
    }
}
```

这里需要特别处理舍入导致的进位，因为进位可能使指数增加。

2.18 P18 bitCount

题目要求：统计一个 32 位整数中有多少个 bit 为 1。

采用分组统计的方法，从相邻的 1 bit 开始逐渐扩大统计范围：

```
m1=0x55;
m1=m1+(m1<<8);
m1=m1+(m1<<16);
```

```
tmp1=(x&m1)+((x>>1)&m1);
```

然后依次统计 4 bit、8 bit、16 bit：

```
tmp2=(tmp1&m2)+((tmp1>>2)&m2);  
tmp3=(tmp2&m3)+((tmp2>>4)&m3);  
tmp4=(tmp3&m4)+((tmp3>>8)&m4);
```

最后合并得到总的 1 的数量：

```
return (tmp4&m5)+((tmp4>>16)&m5);
```

2.19 P19 bitReverse

题目要求：将 32 位整数的 bit 顺序完全反转。

最终采用分层交换的方法，而不是逐个 bit 移动。首先构造不同阶段的掩码：

```
int m1,m2,m3,  
    m4=0xff+(0xff<<16),  
    m5=0xff+(0xff<<8);  
  
m3=m4^(m4<<4);  
m2=m3^(m3<<2);  
m1=m2^(m2<<1);
```

然后依次交换相邻的 1 bit、2 bit、4 bit、8 bit，最后交换高低 16 bit：

```
x=((x>>1)&m1)+((x&m1)<<1);  
x=((x>>2)&m2)+((x&m2)<<2);  
x=((x>>4)&m3)+((x&m3)<<4);  
x=((x>>8)&m4)+((x&m4)<<8);  
  
return ((x>>16)&m5)+(x<<16);
```

相比逐位反转，这种方法将问题转化为多轮分组交换，在较少操作数下完成整个 32 位的反转。

本部分总结

这 19 道题从简单的位构造和基本逻辑运算开始，逐渐扩展到移位、掩码、补码、溢出判断、舍入以及 IEEE 754 浮点数表示。前面的题目主要帮助熟悉 bit-level 操作，后面的题目则要求将数学问题转化为对二进制位模式的操作。通过这些题目，可以看到许多看似不同的问题实际上都可以归结为对符号位、有效位、掩码以及移位关系的分析，从而在受到运算符和操作次数限制的情况下完成计算。

3 实验总结与心得体会

通过本次 DataLab 实验，我对整数的补码表示、位运算、移位以及 IEEE 754 浮点数表示有了更加具体的理解。最终 19 个函数均通过操作数检查，功能测试结果为：

Total points: 110/110

这次实验让我比较明显地感受到，bit 级别的位运算思维和日常直接进行数学计算的思维方式并不一样。很多问题如果直接按照数学公式去做，会遇到运算符限制或者整数溢出等问题；换一个角度，从二进制位本身出发，反而能够找到更加直接的解决方法。尤其是在一些关键问题中，异或运算往往能够起到很好的作用。

准备这个 Lab 的老师 and 助教们辛苦了，也感谢课程在题目设计和测试方面提供的帮助。